Temporal TV Recommendation on SWEET.TV

Petr Vorozhtsov | Individual | 2026-10-24

Email: pavorozhtsov@edu.hse.ru | Assignment: LLM4Rec Midterm Quiz (GSB)

Kaggle: PetrVorozhtsovLMS

Repository: RecSys-LLMs / exam-sweettv / exam_sweettv

Abstract

SWEET.TV requires five ranked programme recommendations per test user from viewing logs whose schedule labels are sparse. This study joins viewing intervals to labelled broadcasts and compares content-based filtering and association scoring with a popularity reference on a temporal hold-out. Popularity achieves local MAP@5 of 0.162263, compared with 0.157865 for association and 0.008688 for content, while the submitted association file returns a Kaggle private score of 0.01454. Popularity is the strongest measured launch candidate, and the submitted personalised model requires further work on cold-start evaluation, metadata timing and association normalisation.

Index Terms: recommender systems; implicit feedback; temporal validation; association rules; content-based filtering.

1. Introduction

The prediction target is a ranked Top-5 list of programme IDs for each of 1,260 users in weeks 31-42. Earlier viewing intervals identify implicit interactions, not explicit satisfaction. Only about 10.8% of schedule rows have a programme ID [1], so matching quality and coverage constrain every recommender before ranking begins.

For example, an interval on channel c is matched to a labelled broadcast on the same channel; its overlap seconds enter the user profile. For user 8377619604347126107, the submitted association list starts with 2500413 (Civil Defence). This illustrates how an interval-level log becomes a ranked programme-level result. Contributions are a reproducible join-and-ranking pipeline, a shared temporal comparison of two personalised methods plus baseline, and an evidence-based analysis of two fixed users.

2. Related work and alternatives

Cosine similarity compares normalised feature vectors [2], matching the content implementation. Association confidence and lift describe conditional co-occurrence [3]; the current lift-like weighting differs from the classical definition, as documented below. Personalised PageRank propagates preference through a graph [4], while implicit ALS learns low-rank factors from confidence-weighted interactions [5]. These references were consulted during the report audit; scikit-learn, mlxtend, NetworkX and implicit were not dependencies of the submitted pipeline.

Alternatives considered: implicit ALS could share statistical strength across sparse histories but needs tuning of factors and confidence weights; personalised PageRank could exploit multi-hop co-view paths but adds graph construction and convergence choices. Neither was implemented or benchmarked, so no superiority over the measured methods is claimed. Content and direct pair scoring were chosen for their transparent features and bounded implementation cost.

Study scope

The submitted file remains the association solution. The final selector compared association with content, excluding popularity from selection. This explains the submitted choice but does not justify claiming it is best overall. The launch recommendation in Section 5 follows the baseline result instead.

3. Method

Pipeline: read labelled schedules -> retain template users -> join intervals -> aggregate user-item overlap -> fit on weeks 11-20 -> evaluate weeks 21-30 -> combine user histories for final ranking -> write five IDs per user. No test-period viewing behaviour is supplied or used. Validation interactions are not used to fit the local popularity, profile or pair models.

3.1. Join and data coverage

dataset11-30.csv contains 5,821,987 views. vsetv_id is matched to channel_id. Viewing bounds come from start_time and stop_time; duraton is viewing length, not programme duration. Blank and zero programme IDs are dropped and cannot become items. For equal channels, overlap = min(stop_view, stop_show) - max(start_view, start_show). A view counts when overlap is at least 60 seconds and overlap / schedule_duration is at least 0.10. The eligible broadcast with maximum overlap is selected; repeated matches accumulate seconds per user-item pair.

The absolute threshold rejects brief switches; the fractional threshold accounts for programme length. Compared with the 396-second median view reported in the assignment, 60 seconds is a modest absolute cutoff. Neither threshold was tuned. A long session spanning several broadcasts loses all but one match; long programmes may fail the 10% condition. This is a known trade-off, not an empirically optimal join.

Processing audit	Count
Input views / excluded non-template views	5,821,987 / 3,648,674
Unmatched or unlabelled / matched	1,671,100 / 502,213
Train-history users / combined-history users	1,250 / 1,258
Final catalogue / submission users	9,671 / 1,260

Matched intervals are 23.11% of the 2,173,313 retained views, not 23.11% of all schedule rows. The audit counts intervals, not distinct user-item pairs. Restricting all model fitting to template users reduces the collaborative signal available from the wider audience.

3.2. Ranking and configuration

Method	Actual scoring rule and parameters
Popularity	Rank by distinct training viewers per ID. Ties follow input-dependent Counter ordering.
Content	Category weight 2; genre 1; production decade 0.5; actor/director tokens longer than two characters 0.25. User vector is the sum of item vectors weighted by log(1 + overlap_seconds); rank by cosine similarity, then string ID.
Association	Build co-view pairs from the first 60 distinct IDs in insertion order per user. confidence(a,b) = pair_count(a,b) / item_users(a). Contribution: log(1 + overlap_user_a) * confidence(a,b) * log(1 + lift_like).
Lift-like term	confidence(a,b) / (item_users(b) / sum(item_users.values())). The denominator is total item support, not the number of users; this is not classical lift [3].
Output	Scan the method ranking, then popularity; remove previously watched IDs and within-list duplicates; stop at five IDs. Scores are rankings, not interchangeable probabilities.

3.3. Candidates, repeats and cold start

Validation candidates combine labelled IDs from weeks 11-20 and 21-30. Final candidates combine weeks 11-42, rather than restricting to test-period airings. Watched IDs are blocked identically across methods, although validation truth retains repeat views. For empty histories, the submitted association ranking is empty and returns the Top-5 popularity list; popularity does the same. The current content code instead returns five zero-score candidates in string-ID order. Thus the required identical fallback across all methods is not fully satisfied; this limitation is disclosed rather than hidden.

4. Experiments and verification

Protocol: train on weeks 11-20 and validate on weeks 21-30; K = 5 throughout. This forward split reflects the later test window. AP@5 sums Precision@r at relevant ranks and divides by min(5, number of relevant IDs); MAP@5 averages across users with nonempty validation truth. Users absent from train remain in the denominator but receive empty predictions, a cold-start evaluation defect shared across the reported methods.

Method	Temporal validation MAP@5	Difference from popularity
Popularity	0.162263	Reference
Association	0.157865	-0.004398
Content	0.008688	-0.153575

Association is approximately 2.71% below popularity relative to the baseline score. Content is much weaker. This supports popularity as the next launch baseline; it does not establish the causes of the gap. No confidence intervals, hyperparameter sweep or ablation were performed. The association submission has Kaggle Private Score 0.01454 and Public Score 0.01740; local and Kaggle values use different windows and targets and are not directly comparable.

4.1. Environment and measured evidence

Windows; Python 3.12.14; CPU identifier: AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD; 16 logical CPUs; 15.37 GiB installed physical memory. run.py uses only the Python standard library. PDF generation uses ReportLab 4.4.9; rendering checks use Poppler. No random seed or PYTHONHASHSEED was set. No stochastic optimiser is used, but set iteration can affect insertion order and tied association scores; exact cross-process reproducibility is not assumed.

Independent audit rerun of the unchanged pipeline: 511.10 seconds wall-clock, exit code 0. Output comparisons: submission.csv: byte-identical; validation_results.json: byte-identical; audit.json: byte-identical; examples.json: byte-identical. This is a measured rerun, not the unrecorded original runtime. See outputs/reproduction_verification.json and reproduction.log.

Verification script verify_report.py passed: exact template header and user order, 1,260 rows, exactly five distinct IDs per row, and six example lists of five IDs with no overlap with their saved user histories. The checks also independently compared the code metric with this worked example: relevant {A,C}, predictions [A,B,C,D,E], AP@5 = (1 + 2/3) / 2 = 0.8333333333. This is an assistant-run check; the student has not confirmed personally hand-computing it.

5. Discussion: strategy, failures and improvement

My platform choice: I would begin with popularity because it has the highest observed local MAP@5, low computational cost and a simple reserve for users without history. I would test association next as a transparent personalisation candidate; its current submitted score is not evidence that it beats popularity. This first-person explanation was drafted with AI assistance and remains subject to my own review.

What worked: the pipeline produced a scoreable Top-5 file, both required personalised methods were evaluated, and user-specific lists can be traced to exact IDs. What did not work: content failed to match the baseline; association preserved all five baseline IDs for the second example user; and a cold-start probe exposed inconsistent content behaviour. Popularity is cheap to aggregate; content scans candidates per user; pair construction is capped at 1,770 unordered pairs per user before aggregation.

Observed failure and diagnostic: an empty content profile returns [1,2,3,4,5] on the audit toy catalogue rather than the prescribed popularity list [10,20,30,40,50]. The cause is zero cosine scores sorted by ID. A proposed fix is to route empty or zero-norm profiles directly to popularity and generate validation predictions for every validation user. This model fix has not been applied or benchmarked; the report fix is to remove the earlier false claim of a shared fallback. The failing behaviour was confirmed in outputs/report_verification.json.

Next change: repair the common cold-start branch and rerun the same hold-out before changing another variable. Separately, freeze metadata at the training cutoff: the current content features merge future-schedule attributes, which exceeds strict availability-only access. The association pair table also remains trained on weeks 11-20 while final profiles use weeks 11-30. These are unresolved implementation limitations; no full-compliance claim is made.

5.1. Two users: recommendations against observed histories

The first two template users are a reproducible, non-cherry-picked choice; both have usable histories with different category mixes. They are not representative of all users or cold start. Histories cover weeks 11-30; future truth is unavailable. English titles are display translations from schedule metadata; IDs retain source identity.

User 8377619604347126107: 90 historical programmes

Historical category counts: Films: 35; Entertainment: 19; Educational: 11; Children: 10; Information: 7; Series: 6; Other: 2. Counts measure unique IDs, not viewing-time shares.

Observed history examples: 200337: Morning with Inter (Entertainment); 12002955: Peppa Pig (Children); 2400467: Window News (Information); 20088: Breakfast with 1+1 (Entertainment).

Method	Ordered Top-5: ID, English title (category)
popularity	1. 2500413: Civil Defence (Information); 2. 700389: Who Is on Top? Season 6, Episode 7 (Entertainment); 3. 10002893: When We Are at Home (Series); 4. 5800509: Real Mysticism (Entertainment); 5. 500281: This Is Ukraine Speaking (Entertainment)
content	1. 90079323: Now You See Me 2 (Films); 2. 90077525: Plastic (Films); 3. 90072921: Big Soldier (Films); 4. 90078430: Kingsman: The Secret Service (Films); 5. 90079464: Deadpool (Films)
association	1. 2500413: Civil Defence (Information); 2. 700389: Who Is on Top? Season 6, Episode 7 (Entertainment); 3. 10003159: Nothing Happens Twice, Episodes 1-2 (Series); 4. 700390: Superintuition, Season 3, Episode 4: Andrey Bednyakov and Yulia Sanina (Entertainment); 5. 500281: This Is Ukraine Speaking (Entertainment)

Content returns five films, aligned with the largest historical category (35 of 90 IDs), but category alignment alone cannot establish relevance. Popularity returns three entertainment shows, one information show and one series; association replaces two of those IDs and shares three of five. These lists reflect additional historical categories (19 entertainment, 7 information, 6 series), while offering unseen IDs within the logged history.

User 8381667675275833309: 53 historical programmes

Historical category counts: Films: 13; Entertainment: 11; Children: 9; Information: 7; Educational: 5; Series: 5; Other: 3. Counts measure unique IDs, not viewing-time shares.

Observed history examples: 2400480: TSN: Week (Information); 400426: Wife Swap (Entertainment); 2500413: Civil Defence (Information); 240081: Facts: Day (Information).

Method	Ordered Top-5: ID, English title (category)
popularity	1. 2400508: Today: Results with Oleg Panyuta (Information); 2. 20088: Breakfast with 1+1 (Entertainment); 3. 200337: Morning with Inter (Entertainment); 4. 6200371: Jedi (Educational); 5. 400335: A House for Dad, Season 7, Episode 1 (Entertainment)
content	1. 500368: 60 Minutes with Olga Skabeyeva and Evgeny Popov (Entertainment); 2. 90082757: Bernie the Dolphin (Films); 3. 90082758: Bernie the Dolphin 2 (Films); 4. 90076672: Courier from Paradise (Films); 5. 90081355: The Extraordinary Journey of the Fakir (Films)
association	1. 2400508: Today: Results with Oleg Panyuta (Information); 2. 20088: Breakfast with 1+1 (Entertainment); 3. 200337: Morning with Inter (Entertainment); 4. 400335: A House for Dad, Season 7, Episode 1 (Entertainment); 5. 6200371: Jedi (Educational)

Content returns four films and one entertainment show; films are the largest historical category (13 of 53 IDs). Popularity and association share all five IDs, with the last two positions exchanged: both contain three entertainment, one information and one educational programme. Those categories occur in the history (11, 7 and 5 IDs). For this user the association model changes ordering rather than expanding the recommended set.

All six lists contain five IDs, with no ID in the corresponding saved history. This follows from the explicit filter, not from proof of useful discovery. Familiar categories can support plausibility, but an unfamiliar ID may still be a poor recommendation; the log does not show enjoyment or viewing on other platforms. Future per-user hits cannot be checked here.

5.2. Why the approaches differ

Association represents explicit pairwise co-view evidence and conditional direction through confidence. Low-rank factorisation [5] shares information through latent factors and need not preserve an individual observed rule; association is not inherently more expressive and cannot replace latent generalisation. Content can match category, genre, people and decade even when a programme has no co-view links, but can overconcentrate on broad attributes. Personalised PageRank [4] would spread preference across multiple graph steps and redistribute mass through restart, potentially reaching programmes without a direct pair link to a watched ID. Neither ALS nor PageRank was evaluated here, so these are mechanism comparisons, not measured rankings. Popularity already works well both in aggregate MAP@5 and in the second user’s shared Top-5 set.

6. AI usage disclosure and submission evidence

AI tool: Codex desktop assistant. The exact model revision and desktop build were not recorded and are not inferred. AI assisted with planning, code generation, debugging, algorithm explanations, report drafting, translation and verification. No external model was used for ranking. Python supplies the runtime [6]; ReportLab generates the report [7].

Personally performed by the student: Kaggle sign-in and rule acceptance were confirmed in chat; the student uploaded the final CSV and supplied the score screenshot. Assistant checks: code-to-JSON metric comparison, worked AP@5 example, structural CSV assertions, two-user history checks, source-document review, cold-start probe and visual PDF inspection. The first-person strategy paragraph was drafted at the student's request; this does not establish personal verification of every claim.

Trusted or not independently established: attribution of the supplied score screenshot to the designated file relies on the student; hidden test labels, enjoyment and business effects are unavailable. No student hand-calculation confirmation or personal rewriting of the discussion has been recorded. The student should review the calculation and reasoning before submission; AI cannot perform that personal requirement on their behalf.

Session audit trail: submit the original, unmodified native export for this chat, identified by rollout-2026-10-24T15-33-29-01a16de6-e133-75f3-9079-c31978825d33.jsonl. Native JSONL is retained as exported; a reformatted JSON summary is not a replacement. Use a fresh export that includes the final edits. The export is separate from this PDF and should not be altered to remove or rewrite events.

Kaggle returned score

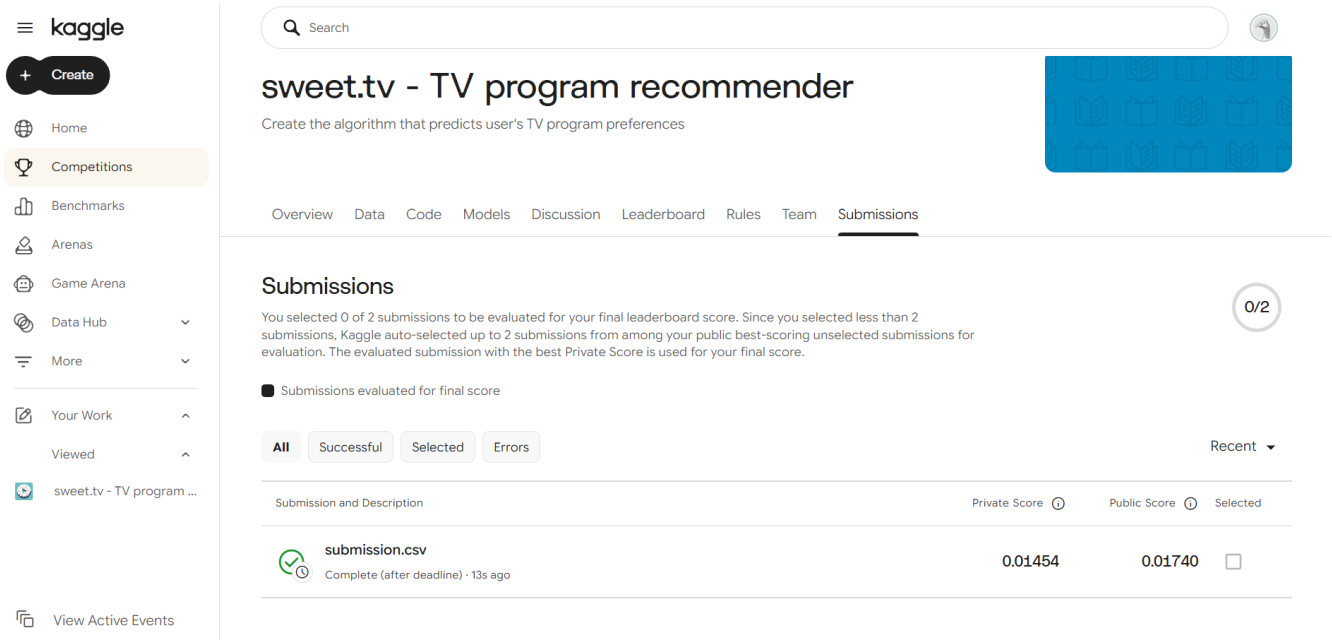


Figure 1. Student-supplied Submissions screenshot: submission.csv, Complete (after deadline), Private Score 0.01454, Public Score 0.01740. Cohort grading uses returned private score, not position on the frozen leaderboard.

Submission package and remaining requirements

The course channel receives exactly two files: this PDF and the original AI-session export. The repository contains run.py and outputs/submission.csv used for the reported score. The report supplies all required comparison and analysis topics, but the implementation has the limitations disclosed in Sections 3-5. Personal verification and review remain the student's responsibility. The guidelines contain both five- and six-page limits; this revision uses five main pages plus references to satisfy the stricter reading.

References

Sources [2]-[5] were consulted for definitions and alternatives during this report audit, not used as software in run.py. All web pages below were opened on 2026-10-24; no experimental claim is attributed to them.

[1] S. Jin, "Final Exam Quiz (GSB): TV Program Recommendation on sweet.tv," course assignment, 2026; "Midterm Quiz," supplied slides; and "LLM4Rec - Homework Guidelines," supplied course guidelines. Local sources: exam_quiz_GSB.md, LLM4Rec. Midterm Quiz.pdf, and homework_guidelines.pdf.

[2] scikit-learn developers, "cosine_similarity," official API documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html. [Accessed: 2026-10-24].

[3] mlxtend contributors, "Association rules," official user guide, metrics section. [Online]. Available: https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/. [Accessed: 2026-10-24].

[4] NetworkX developers, "pagerank," official API documentation, personalization and damping parameters. [Online]. Available: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.link_analysis.pagerank_alg.pagerank.html. [Accessed: 2026-10-24].

[5] Implicit contributors, "AlternatingLeastSquares," official CPU model documentation. [Online]. Available: https://benfred.github.io/implicit/api/models/cpu/als.html. [Accessed: 2026-10-24].

[6] Python Software Foundation, "time - Time access and conversions," official documentation, perf_counter. [Online]. Available: https://docs.python.org/3/library/time.html#time.perf_counter. [Accessed: 2026-10-24].

[7] ReportLab, "Chapter 1: Introduction," official user guide. [Online]. Available: https://docs.reportlab.com/reportlab/userguide/ch1_intro/. [Accessed: 2026-10-24].